

ALUMNEX CONNECT

TECHNICAL ARCHITECTURE & SYSTEM DESIGN SPECIFICATION

A High-Performance, Real-Time MERN Stack Alumni Networking Platform

EXECUTIVE PROJECT OVERVIEW

Alumnex Connect is a premium, enterprise-grade full-stack networking platform engineered to bridge the professional and educational gap between active students and university alumni. Built upon the robust and modern **MERN (MongoDB, Express, React, Node.js)** stack, the system is designed to handle real-time bi-directional communications, provide automated AI career mentorship, and integrate multi-platform performance tracking into a seamless glassmorphism user interface.

This technical specification acts as a comprehensive reference guide for the project review board and system architecture interviewers. It analyzes the system's distributed architecture, database schemas, asynchronous multi-platform APIs, real-time message exchange logic, and deployment methodologies.

Architectural Metric	Implementation Details
Project Name	Alumnex Connect (Full-Stack Alumni Platform)
Primary Stack	React 18 · Node.js · Express.js · MongoDB Atlas (Mongoose)
Real-Time Protocol	Socket.IO (WebSockets with server-side in-memory mapping)
External APIs	Groq AI (Llama 3), Remotive Jobs, Codeforces, LeetCode, GitHub, GFG, HackerRank
Security Paradigm	JWT Stateless Authentication, Bcrypt Hashing, OTP Email Verification
Optimization Layers	In-Memory Caching (Node-Cache), Parallel Query Aggregation, DB Compounding

Key Design Note: This document has been compiled directly from production-level codebase documentation. All modules described are fully functional and equipped with end-to-end routing, validation, and real-time triggers.

1. SYSTEM ARCHITECTURE & INTEGRATION MAP

The Alumnex Connect architecture follows a decoupled, client-server model designed to maintain operational resilience, stateless scalability, and immediate real-time event updates. The system segregates responsibilities into distinct layers, allowing independent scaling of client connections and persistent data transactions.

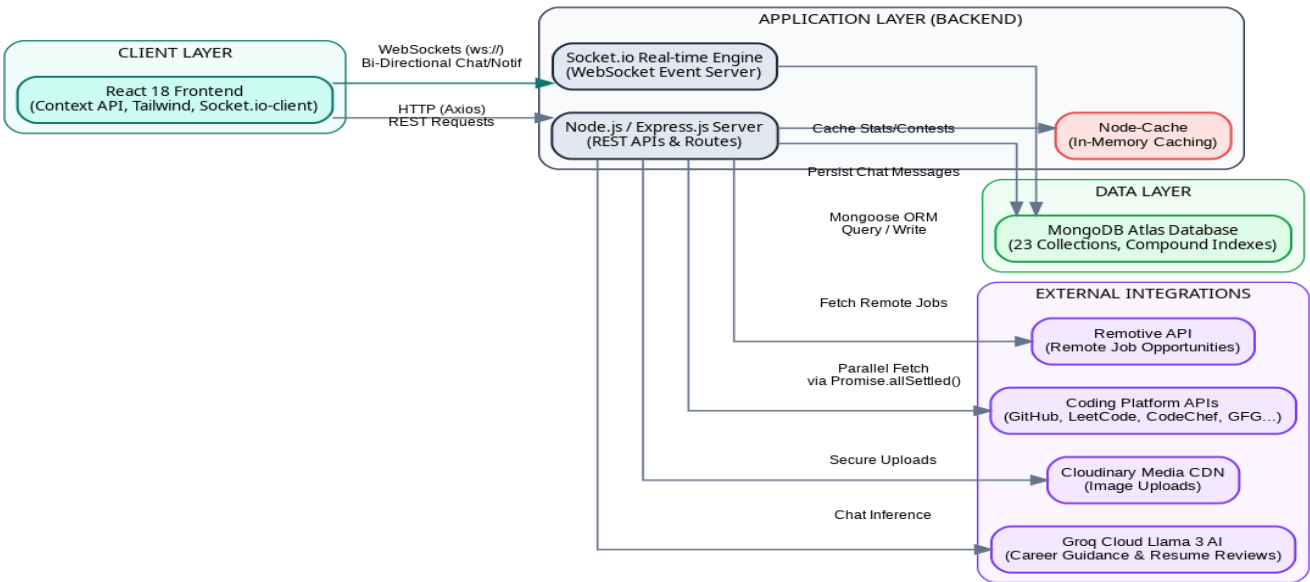


Figure 1.1: Unified High-Level Architecture and Service Integrations Map.

1.1 Layer-by-Layer Architectural Analysis

- 1. Client Layer (React 18):** Constructed with a single-page architecture using React Router v6 for dynamic routing, Context API for persistent global authentication state, and a modern glassmorphism Tailwind CSS aesthetic. It maintains an active TCP session with the WebSocket server via the Socket.io-client library.
- 2. Application Server Layer (Node.js & Express):** A cluster of stateless Express.js HTTP route handlers responsible for user operations, mentorship booking, forum management, and API proxy routing. It utilizes asynchronous pipelines to query external microservices and databases.
- 3. Real-Time Engine (Socket.IO):** Operating on top of the same HTTP server port, the Socket.IO server manages persistent, bi-directional client channels. It coordinates chat messaging, global live notifications, and online presence tracking across servers.
- 4. Data Store Layer (MongoDB Atlas & Node-Cache):** Highly optimized transactional storage utilizing Mongoose schemas. It features robust compound indexing to prevent full-table scans. Frequently fetched external data is cached in-memory with a local TTL cache.

2. REAL-TIME ENGINE & WEBSOCKET TOPOLOGY

A key differentiator of Alumnex Connect is its high-performance, event-driven real-time system. Instead of exhausting resources on periodic HTTP polling, the platform utilizes WebSockets via Socket.IO. This allows the server to instantly push data to active clients when database changes occur, reducing latency to virtually 0ms.

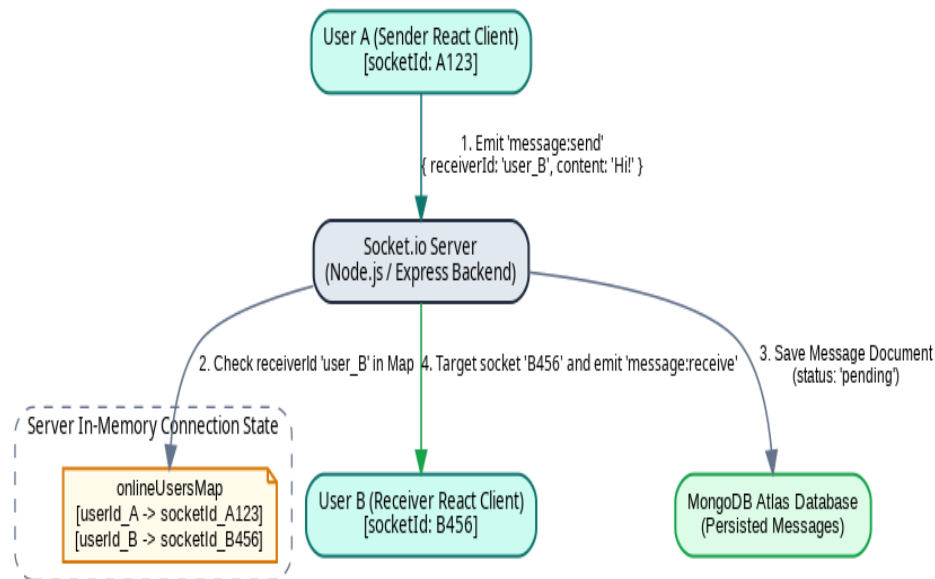


Figure 2.1: Real-Time Event Emit Lifecycle & In-Memory Socket State Tracking.

2.1 Stateful Socket Management & Room Strategy

To target specific users across a cluster of connections, the server maintains an in-memory **onlineUsersMap** connecting unique database userIds to active socketIds. When a client establishes a socket session, the server immediately performs JWT authentication and adds them to the map. On disconnect, the reference is cleanly pruned, broadcasting status changes across the system.

Socket Event	Source	Technical Action & Operational Lifecycle
message:send	Client	Emits content, receiverId, and media metadata to the socket server.
message:receive	Server	Targets the receiver's socketId from onlineUsersMap and emits the new message payload.
group:message	Client	Emits a message containing a ChatGroupId to a specific room.
group:message:receive	Server	Broadcasts the incoming payload to all active sockets joined in the ChatGroup room.
notification:received	Server	Real-time push notification for mentorship requests, job matches, and forum replies.
job:new	Server	Broadcasts to all connected students when an alumnus creates a new job or referral posting.

Key Design Note: To ensure high scalability, group chats utilize native Socket.io rooms (socket.join(groupId)). The server avoids iterating through membership lists in JavaScript; instead, it delegates broadcasting to the native room layer.

4. DEVPULSE INTEGRATION & CONCURRENCY SYSTEM

The **DevPulse Module** represents one of the most high-performance sections of the codebase. It consolidates real-time programming statistics and achievement badges from external platform APIs (GitHub, LeetCode, HackerRank, GFG, CodeChef, and Codeforces) into a single unified profile dashboard.

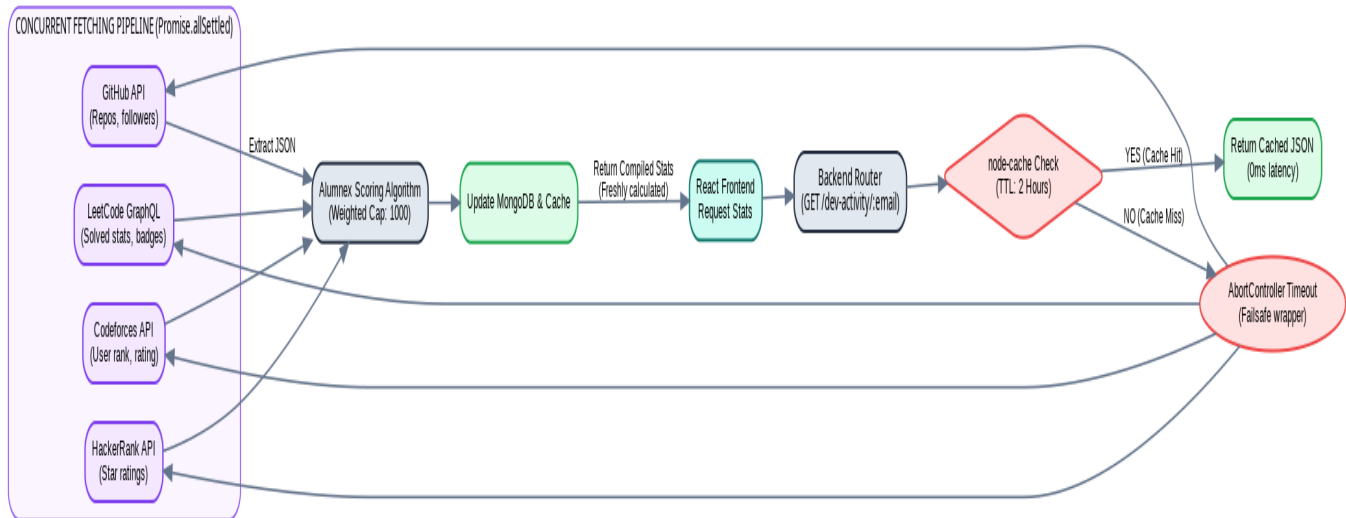


Figure 4.1: Asynchronous Parallel API Aggregation and Caching Architecture.

4.1 Concurrent Fetching & Server Resilience Design

Fetching stats from six separate external platforms sequentially would lead to severe latency bottlenecking. Instead, the backend initiates parallel requests using `Promise.allSettled()`. This allows all HTTP connections to be negotiated concurrently, bounding the request time to the single slowest API.

To prevent server threads hanging due to external rate-limiting or server downtime, each fetch request is wrapped with an **AbortController** timeout of 5 seconds. If a platform fails to respond within the limit, the pipeline cleanly isolates that failure, allowing other successful platform stats to populate the profile.

4.2 DevPulse Algorithmic Scoring & In-Memory Caching

Once statistics are parsed, the backend executes the **Alumnex Scoring Algorithm**. This mathematical scoring system calculates a relative developer rating capped at 1000 points, weighting LeetCode difficulty distributions, HackerRank star counts, GFG score metrics, and public GitHub repository output. To ensure 0ms latency on subsequent views, compiled payloads are cached for 2 hours using **Node-Cache**. Any request within this window bypasses external APIs completely, preventing API key rate exhaustion.

5. DATABASE MODEL & LOGICAL SCHEMA DESIGN

The data layer is constructed on top of **MongoDB** using **Mongoose schemas**. The database contains 23 core collections. Below are detailed relational schema breakdowns of the main models essential for professional system review.

5.1 User Document Schema Model

Field Name	Data Type	Schema Role & Validation Logic
_id	ObjectId	Primary key identifying the user account.
name / email	String	User's full name and unique verified email address.
password	String	Cryptographically hashed password (using bcrypt salt).
role	Enum	Restricted roles: 'student', 'alumni', or 'admin'.
isVerified	Boolean	OTP verification flag. Controls platform access.
skills / interests	Array[String]	Used for mentorship and search indexing.
connections	Array[ObjectId]	Bidirectional references to other User documents.

5.2 DevProfile Schema Model (Coding Integrations)

Field Name	Data Type	Schema Role & Validation Logic
user	ObjectId	Required reference to the parent User document.
username	Object	Linked platform usernames: { github, leetcode, gfg, hackerrank, codeforces, codechef }.
stats	Object	Cached JSON stats retrieved from external API pipelines.
alumnexScore	Number	Calculated rating score capped at 1000.
lastUpdated	Date	Timestamp of last fetch. Used to enforce 2-hour caching TTL.

5.3 Job & JobApplication Schema Models

Field Name	Data Type	Schema Role & Validation Logic
title / company	String	Details of the career opportunity.
type	Enum	Restricted values: 'Full-time', 'Part-time', 'Internship', 'Contract'.
postedBy	ObjectId	Ref to User (Alumni only) who created the job posting.
applicantId	ObjectId	Ref to User (Student applicant) inside JobApplication document.
resumeLink	String	Secure PDF link pointing to applicant's resume storage.
status	Enum	Application status: 'pending', 'reviewed', 'accepted', or 'rejected'.

6. PERFORMANCE OPTIMIZATION & DEPLOYMENT PIPELINE

To handle scaling and ensure high availability, several advanced system optimizations and deployment pipelines are implemented. These details demonstrate a strong understanding of web scalability during technical interviews.

6.1 Production-Level Indexing Strategy

Without compound indexing, database lookups on frequently queried relational fields turn into full-collection scans. In a production system, this causes extreme CPU bottlenecking. To prevent this, compound and unique indexing has been implemented on MongoDB Collections:

MongoDB Schema	Applied Indexes	Interview Justification & Operational Lifecycle
User Schema	email_1	Unique index to enforce zero-duplicate registration logic.
Message Schema	sender_1_receiver_1_createdAt_-1	Compound index enabling immediate real-time rendering of sequential chat history.
Job Schema	postedBy_1_isActive_1	Allows instantaneous filtering of active opportunities without scanning expired data.
DevProfile Schema	user_1_alumnexScore_-1	Compound index for high-speed Leaderboard queries with pre-sorted rankings.

6.2 Cloud Services & Deployment Flow

1. Backend Hosting: Deployed on **Render Cloud** on top of a Node.js runtime environment. A fallback background runner maintains database sync routines.

2. Frontend CDN: React build static assets are compiled and hosted on **Vercel / Netlify** CDN, minimizing time-to-first-byte (TTFB) for users worldwide.

3. Media CDN: Profile pictures, chat attachments, and PDF resume links bypass database servers. They are dispatched securely to **Cloudinary** via secure HTTPS presigned upload streams, optimizing load times.

```
# Deployment Flow & Package Management
$ cd backend && npm install
$ npm run build
$ pm2 start server.js --name "alumnex-prod" # Run backend daemonized
```

7. SYSTEM ENDPOINTS & API CATALOGUE

A core pillar of clean MERN system design is a restfully consistent, strictly validated routing table. Below is the consolidated system REST API catalogue covering user, profile, career, and coding integrations.

HTTP Method	Express Route Path	Service Layer Action & Execution Details
POST	/api/auth/register	Registers new users. Triggers Bcrypt hashing & nodemailer OTP email.
POST	/api/auth/login	Authenticates credentials, returns stateless JWT and session data.
POST	/api/auth/verify-otp	Verifies 6-digit OTP code, transitioning account verified state to true.
GET	/api/dev-activity/:email	Fetches parallel programming stats (cached 2hr via Node-Cache).
POST	/api/dev-activity/verify-platform	Validates coding platform profile ownership using bio verification codes.
GET	/api/jobs	Retrieves local posted jobs merged with external Remote API jobs.
POST	/api/jobs/:id/apply	Submits student application. Emits Socket.io alert to job poster.
POST	/api/messages/send	Saves direct messages and fires real-time Socket.io message triggers.
GET	/api/leaderboard	Queries DevProfile collection, returning users sorted by Alumnex Score.
POST	/api/mentorship/request	Submits mentorship request. Dispatches real-time global notification.

Key Design Note: All API routes are protected by JWT Bearer middleware and utilize strict Express error validation bounds. Payload structures are strictly validated before MongoDB operations are committed.